

TP 5 — Convertisseurs de bases · Mystères des flottants

Semaine 5 ■ Fichiers dans NSI/TP05/ ■ 2 h

Objectifs

- Programmer soi-même les conversions décimal $\leftrightarrow$ binaire
- Vérifier ses fonctions avec `bin()`, `hex()`, `int(..., 2)`
- Constater expérimentalement les limites des flottants

Partie A — Découverte des outils Python (console)

(15 min)

Tester et noter : `bin(22)` · `hex(47)` · `int("10110", 2)` · `int("2F", 16)` · `0b101 + 0x0A`.
Que renvoie `bin(...)` : un nombre ou une chaîne ?

Partie B — Écrire ses convertisseurs — bases.py

(45 min)

(Interdiction d'utiliser `bin/hex/int(..., 2)` dans les fonctions — ils ne servent qu'à vérifier !)

1. `binaire_vers_decimal(mot)` — prend une chaîne comme `"10110"`, renvoie l'entier. Méthode : parcourir les caractères ; pour chaque caractère, `resultat = resultat * 2 + int(caractere)`. (Parcours d'une chaîne : `for c in mot:` — vu en détail au ch. 4.)
2. `decimal_vers_binaire(n)` — renvoie la chaîne binaire, par divisions successives (concaténer les restes à gauche : `reste + resultat`). Cas particulier : que renvoyer pour `n = 0` ?
3. Vérifications croisées :

```
print(binaire_vers_decimal("10110") == 22)
print(decimal_vers_binaire(22) == "10110")
for n in range(200):      # test en masse !
    if binaire_vers_decimal(decimal_vers_binaire(n)) != n:
        print("BUG pour", n)
```

4. ★ `decimal_vers_hexa(n)` — même principe avec la chaîne `"0123456789ABCDEF"` (le chiffre de valeur `r` est `chiffres[r]`).

Partie C — Complément à 2 — signe.py

(30 min)

1. `complement_a_2(mot)` — prend une chaîne de 8 bits, renvoie la chaîne représentant son opposé (inverser chaque bit, puis ajouter 1 — on pourra passer par `binaire_vers_decimal` : `opposé = 256 - valeur`, sauf pour 0).
2. Afficher la table des mots 1111 1111 à 1111 0000 avec leur valeur signée.
3. Vérifier que `complement_a_2(complement_a_2(m)) == m` pour quelques mots.

Partie D — Mystères des flottants — flottants.py

(30 min)

1. Faire afficher : `0.1 + 0.2`, `0.1 + 0.2 == 0.3`, `1.1 * 3`, `0.1 * 10 == 1.0`.
2. Écrire `presque_egaux(a, b)` qui renvoie `True` si `abs(a - b) < 1e-9`, et refaire les tests avec elle.
3. **L'expérience qui fait réfléchir** : additionner 0.1 dix fois de suite dans une boucle. Le résultat vaut-il exactement 1.0 ? Afficher la différence.
4. ★ Trouver expérimentalement le plus petit `n` tel que `10.0 ** n + 1 == 10.0 ** n` (boucle `while`). Interpréter : à cette échelle, les flottants ne « voient » plus le +1.